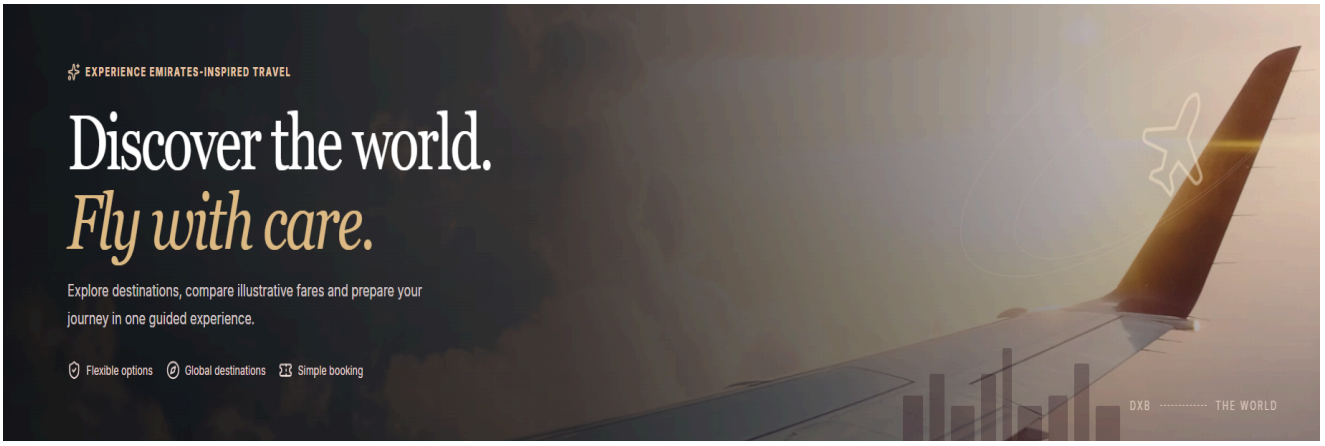


Solution Design & Architecture

EMIRATES-INSPIRED AGENTIC MOCK FLIGHT WEBSITE

Booking Portal • Manage Booking • Web Check-in • Multi-Agent Workflow • Self-Corrective Agentic RAG

Prepared for: Project solution design and code review

Repository: https://github.com/AIstar007/Agentic-Mock-Flight-Website.git

Document version: 1.0 | Date: 12 October 2026

Document Information

Field	Value
Project Name	Agentic Mock Flight Website — Emirates-inspired synthetic passenger journey
Prepared By	Project author: Alen Thomas (as indicated in reference template)
Document Version No.	1.0
Document Version Date	12 October 2026
Title	Solution Design and Architecture
Reviewed By	Pending review
Review Date	Pending review
Repository	https://github.com/AIstar007/Agentic-Mock-Flight-Website.git
Default Branch	main (repository metadata)
Document Status	Draft — technical validation required

Document Statistics

Document Title	Document ID	Template Version	Date of Release	Data Classification	Document Status
Emirates Agentic Flight Website Solution Design	TBD	Created my own template	12 Oct 2026	Internal project documentation; synthetic POC	Draft

Contents

1. Introduction 3
1.1. Scope 3
Integrations: 3
Components: 3
1.2. Out of Scope 4
Definitions, Acronyms and Abbreviations 4
1.3. Summary of Existing Functionality 4
1.4. Requirement Details 5
2. GENERAL OVERVIEW AND DESIGN GUIDELINES/APPROACH 7
2.1. ASSUMPTIONS / CONSTRAINTS / STANDARDS 7
2.1.1. Assumptions 7
2.1.2. Constraints 7
2.1.3. Standards 7
2.2. ALIGNMENT WITH TARGET ARCHITECTURE 8
3. Current Architecture design 9
4 Non-Functional Requirements 14
4.1. Archiving 14
4.2. Logging 14
4.3. Auditing 14
4.4. Scalability 14
4.5. Rate Limit 14
4.6. Performance 15
4.7. System Management and Monitoring 15
4.8. User Role and Access Management 15
5. Risk and Mitigation 16
6. Appendices 16

Important scope boundary
This is a design document for an Emirates-inspired demonstration. It does not claim to be endorsed by Emirates, to call Emirates production systems, to make authoritative immigration decisions, or to issue a valid travel document.

1. Introduction

The Agentic Mock Flight Website is a synthetic airline booking and web check-in proof of concept. It combines an Emirates-inspired user experience with a React/Vite frontend, a FastAPI backend, a LangGraph workflow orchestrator, named specialist agents, a deterministic rule layer, and a self-corrective Agentic RAG assistant for passenger FAQs and error explanations.

The repository covers two connected journeys: (1) flight search and mock booking, including fare choice, passenger/contact details, optional extras, seat selection, review, simulated payment and synthetic confirmation; and (2) Manage Booking / web check-in, including booking lookup, eligibility checks, synthetic passenger-document validation, seat/ancillary confirmation, and a decorative boarding-pass artifact. Page controls and chat actions are designed to update the same application state. Backend validations remain authoritative.

Unlike a production airline system, all customer bookings, flight schedules, fares, seat inventory, payment outcomes and completion artifacts in this POC are synthetic. The adapter names Navitaire and Timatic describe interface boundaries and reference-style contracts, not verified vendor deployments or a claim that Emirates uses those systems.

1.1. Scope

The scope covers front-to-back demonstration behaviour, agent routing, deterministic gate enforcement, FAQ retrieval with evidence evaluation, safe error explanation, configuration, operational persistence options, testing and deployment-oriented documentation.

Integrations

- React 19 + Vite 8 browser application (frontend package engine requires Node.js >=22.12.0).
- FastAPI/Pydantic REST API, with OpenAPI available in development.
- LangGraph orchestration for allowlisted workflow actions and self-corrective Agentic RAG.
- Mock Navitaire adapter for synthetic booking retrieval, seat and ancillary boundaries.
- Mock Timatic-style adapter for synthetic document requirement outcomes.
- Versioned SQLite FAQ corpus; optional local Chroma retrieval.
- Optional approved SearXNG-compatible live search endpoint using HTTPS source allowlists.
- Azure OpenAI-compatible endpoint for optional RAG planning/synthesis; deterministic/source-extractive behaviour is possible when model credentials are absent.
- Optional Redis session/checkpoint persistence; optional MongoDB session/audit TTL stores; optional Neo4j rule graph.
- Optional Sarvam AI speech-to-text/text-to-speech proxy configured server-side.
- GitHub Actions CI for backend tests, frontend build, and secret/dependency checks where configured.

Components

Component	Purpose
1. Browser experience	Emirates-inspired booking and check-in workspaces; shared visible form state; floating bottom-right assistant launcher/panel.
2. FastAPI Gateway	REST endpoints; request validation; session-token protection for stateful passenger steps; limited rate control and security headers.
3. Booking/Commerce Services	Synthetic flight search, fare choice, passenger input, add-ons/seats, simulated approval/decline and synthetic booking confirmation.
4. LangGraph Orchestrator	Routes an allowlisted action to a specialist handler. The graph checkpoint should contain opaque references and metadata rather than raw passenger payloads.
5. Booking Retrieval Agent	Validates PNR/reference plus surname and opens a short-lived session for a synthetic booking.
6. Eligibility Agent	Runs deterministic rule tasks, such as booking status, payment balance, passenger data/age accompaniment, itinerary and timing/provider guard checks.
7. Document Validation Agent	Validates synthetic passport field schema and fans out mock checks per passenger and journey; aggregates FAIL/REVIEW_REQUIRED conservatively.
8. Seat Agent	Lists mock seats and validates passenger/journey assignment; supports explicit skip without silently inventing a seat.
9. Ancillary Agent	Presents catalogue; saves per-passenger options; requires explicit confirmation or skip before completion.
10. Check-in Agent	Rechecks hard gates before producing a clearly labelled decorative mock boarding-pass artifact.
11. FAQ / Self-Corrective RAG	Classifies risk, plans/decomposes queries, retrieves, grades evidence, rewrites weak searches, applies authority/freshness checks, evaluates support, corrects or abstains.
12. Mock provider adapters	Navitaire-like reservation/seat contract and Timatic-style travel-document response contract; never treated as official proof.
13. Session / audit storage	In-memory local fallback and optional Redis/MongoDB TTL persistence; redacted events avoid raw PNR/passport values.

Component	Purpose
14. Knowledge and provenance	Approved/versioned FAQ corpus, official source catalog, rules catalog, optional Neo4j relations, URL/authority allowlists, corpus status.
15. Error guidance	Maps API/page failures to a human-readable reason, affected field/passenger/journey, safe correction and escalation recommendation.
16. Diagnostics and CI	Provider diagnostic endpoints are opt-in/protected and read-only; automated tests exercise mock contracts; CI build and audit checks support release discipline.

1.2. Out of Scope

- Actual Emirates booking, inventory, fare pricing or reservation mutation.
- Actual Emirates identity, loyalty/Skywards, payment gateway, ticketing or departure control system integration.
- Real Timatic query or official visa/immigration/health-document decisions in the user journey.
- Passport authenticity, biometric, OCR, identity or fraud verification.
- Issuance of valid e-tickets or boarding passes; generated artifacts are decorative mock outputs.
- Live flight schedules, availability, seat inventory, refunds, disruption rebooking or flight status unless a separately approved integration is implemented.
- Production security certification, airline/legal approval, aviation compliance sign-off or SLA guarantee.
- Automated permission escalation or allowing an LLM/chat message to bypass deterministic checks.
- Kafka / Redis Streams event backbone; event streaming is explicitly deferred until durable cross-consumer replay is required.
- Claiming Navitaire is Emirates' actual reservation platform; current use is only an adapter boundary.

Definitions, Acronyms and Abbreviations

Term	Description
API	Application Programming Interface
Agentic RAG	Retrieval-augmented generation with query planning, evidence checks, retries/correction and abstention
PNR	Passenger Name Record / booking reference used by the mock lookup
SSR	Special Service Request, represented as synthetic provider requirements
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
POC	Proof of Concept
TTL	Time to Live
RBAC	Role-Based Access Control
STT / TTS	Speech-to-Text / Text-to-Speech
ISO-3166	Country code standard used by the input validators
PASS	Mock gate pass under configured demo checks; not official clearance
FAIL	Negative decision; remains blocked
REVIEW_REQUIRED	Insufficient/ambiguous/high-risk evidence; must not be treated as PASS
MOCK_ONLY	Synthetic provider result or transaction, not a live airline response
LangGraph	Graph-based runtime used to coordinate short allowlisted action workflows and the RAG graph

1.3. Summary of Existing Functionality

The README and architecture notes describe an implementation rather than a blank-slate target. The website already exposes booking, Manage Booking, check-in, help/FAQ, flight-status presentation and chat-assisted navigation. Commerce flow validates dates, origin/destination, trip type, passenger counts, fare selection, passenger/contact fields, seat codes, duplicate assignments and payment simulation. The user can interact by form or by the page-aware chatbot, and assistant actions go through existing API/form handlers rather than directly changing hard gate decisions.

The check-in journey retrieves a synthetic booking using a reference plus surname, runs deterministic eligibility, collects mock passport fields, validates country codes/dates/purpose/duration, performs per-passenger/per-journey mock Timatic-style checks, handles seat and ancillary choices, and generates mock passes only when all required gates pass. Any FAIL or REVIEW_REQUIRED blocks completion and is surfaced with the reason plus repair/escalation guidance.

The Agentic RAG assistant uses a versioned approved source corpus with optional Chroma vector retrieval and optional configured official-site web search. It includes evidence grading, bounded retries, source scope/freshness checks, answer-support evaluation, correction and abstention for high-risk or poorly supported queries. It is not a fixed FAQ string matcher, but the corpus is curated and not a comprehensive country-by-country legal database.

Implementation evidence and limit
Repository validation notes report passing unit suites and frontend production builds on specific revisions, but results differ between README update notes and docs/VALIDATION_RESULTS.md. This document therefore records the latest stated checks as reported, not as independently rerun now; no live Emirates, Timatic, Navitaire, Azure, Redis, MongoDB or Neo4j end-to-end call is claimed.

1.4. Requirement Details

Functional Requirements

1. Booking search and itinerary

Accept origin/destination airport codes, trip type, departure/return dates, adult/child/infant counts and cabin. Reject same airports, past departure, invalid round-trip dates, infant count greater than adults, and passenger totals beyond the documented demo cap.

2. Synthetic flight and fare selection

Return explicitly synthetic flight options and fare families. No request to a real airline inventory or fare quote API.

3. Passenger and contact details

Validate names/allowed punctuation, title, passenger type, DOB, nationality country code, contact email and phone. Apply demo age-category/companion rules and document all local thresholds as POC safeguards.

4. Extras, seats and payment simulation

Allow per-passenger extras and seat selection/skip, validate against mock inventory and passenger/journey identity, review full summary, then simulate approve/decline. QR is non-payable presentation only.

5. Synthetic booking confirmation

Create a mock reference after the simulated outcome passes. The new reference can be looked up within the demo store; it is not a ticket.

6. Manage Booking lookup

Require a well-formed booking reference and surname, normalize input and authorize subsequent workflow steps with a random session token. Real Emirates' public Manage Booking experience similarly asks for reference and last name, but this POC stores synthetic data.

7. Eligibility prechecks

Check confirmed booking, payment balance, journey/passenger shape, time/codeshare/provider constraints and passenger accompaniment in deterministic code. No model is the source of truth for PASS/FAIL.

8. Document input and validation

Accept synthetic passport number, nationality, issuing country, date of birth where required, expiry, purpose and duration. Enforce strict schema, ISO country validation, date parsing and field limits. Do not claim passport authenticity.

9. Per passenger × journey fan-out

Run one document evaluation for each passenger/journey pair, retain decision/code/reasons, and aggregate fail-closed. Any FAIL or REVIEW_REQUIRED blocks the journey from completion.

10. Seat and ancillary handling

List mock seat/ancillary options, validate selected passenger/journey and availability, support explicit selection or skip, and persist selections before completion.

11. Completion and mock boarding passes

Recheck all hard requirements: eligible state, document decision PASS, every required seat/ancillary step confirmed or explicitly skipped according to current flow contracts, and valid session. Output must state "mock / not valid for travel".

12. Agent routing

Route only allowlisted action names through LangGraph/dispatcher. A natural-language assistant can suggest actions; API validators and workflow gate logic independently validate them.

13. Passenger FAQ and self-correction

Plan and decompose broad/compound queries; retrieve from active source corpus and optionally approved HTTPS sources; grade/re-rank, rewrite limited retries, validate authoritative scope/freshness, correct unsupported answers or abstain.

14. High-risk FAQ guard

Visa, transit, passport validity, medical/health-document and country-entry topics require current, specific authoritative evidence. Missing/ambiguous/outdated evidence yields a qualified answer or abstention, not an approval.

15. Page-aware error explanation

For step failure show field/passenger/journey, error code, why it failed, correction path and when to contact official airline/authority. The assistant may not override or mark the gate passed.

16. Voice input/output

Optional Sarvam STT/TTS through backend only; voice and typed utterances share the same allowed-action handler. No API key may be bundled in Vite client variables.

17. Persistence and expiry

Local memory is a single-process development fallback. Optional Redis/MongoDB sessions/checkpoints and TTL audit storage must fail explicitly if configured but unavailable; process-local locks are not multi-replica transactional guarantees.

18. Read-only provider diagnostics

When explicitly enabled with a protection token, optional diagnostics may perform authorized staged reads and document-check simulation in the reference workflow. They must not submit document updates, change seats/extras, complete check-in or issue passes.

19. Audit and redaction

Record timestamps, component, action, outcome, correlation ID and synthetic session fingerprint. Never retain raw passenger names, PNRs, passport numbers, credentials, request prompts with sensitive data or authorization tokens in audit events.

20. CI and validation

Run backend tests, compile checks, frontend build, secret scan, dependency auditing and test scenarios. State exactly what ran; do not infer production certification from unit tests.

2. GENERAL OVERVIEW AND DESIGN GUIDELINES/APPROACH

The system uses a layered client/API/workflow architecture. React presents the journey and chat assistant; FastAPI validates requests and manages session authorization; the LangGraph dispatcher invokes one specialist operation for each accepted action; deterministic Python services enforce the high-consequence workflow invariants; adapters isolate mock provider contracts; and the RAG workflow is a separate evidence-gated path for informational questions. The browser uses the same UI state for form actions and assistant actions, while backend state/gates remain authoritative.

The core action path is a short graph invocation keyed to the session. The repository architecture notes explicitly say raw request payloads and agent results are held in one-call process memory and only opaque references, allowlisted action names and random session identifiers should enter graph checkpoint state. This avoids creating a second long-lived copy of passenger PII in the checkpoint store.

The RAG path is deliberately separated from eligibility. Informational responses may explain public guidance and page errors; no RAG result is accepted as proof of travel-document compliance or a replacement for live official Timatic/immigration decisions. The architecture is therefore not “LLM decides whether passenger may fly”; it is “deterministic gates decide mock workflow progression, evidence-grounded assistant explains policy with abstention when uncertain.”

2.1. ASSUMPTIONS / CONSTRAINTS / STANDARDS

2.1.1. Assumptions

- The application is a synthetic demonstration and uses invented bookings, schedules, seat inventory and payment outcomes.
- The frontend origin is configured through FRONTEND_ORIGIN; deployment also needs VITE_API_BASE_URL pointed at the reachable HTTPS backend.
- MOCK_MODE defaults to true and real provider integrations are not automatically enabled by setting diagnostics credentials.
- The active FAQ corpus and source catalog are maintained as versioned assets and refreshed through documented ingest/activation tasks.
- Azure OpenAI model access is optional for RAG synthesis; when not configured, deterministic planning/source-extractive output may be used.
- Redis/MongoDB/Neo4j are optional; a local in-memory mode is not a durable multi-instance session architecture.
- Provider diagnostic credentials require separate approved access and a coherent provider environment; fixtures/mocked transports do not prove live interoperability.
- Emirates public webpages are reference sources for UX/rules research and can change; current deployments need dated revalidation.
- The exact Emirates production contracts, SDKs, identity model, DCS behaviour and internal policies have not been provided and must not be inferred.

2.1.2. Constraints

- No live Emirates inventory, booking mutation, ticketing, payment, operational flight status or check-in transaction.
- No live Timatic call from the consumer workflow; mock adapter values are not official travel eligibility.
- The local FAQ corpus currently covers 37–40 reviewed docs depending on the active version, as reported by repository notes; it is not exhaustive across jurisdictions.
- Evidence verification is described by the code as conservative heuristic/support checking, not a formal proof of entailment or a guarantee that a public source is legally sufficient.
- External web search works only when an approved reachable endpoint is configured; local/offline mode may abstain on unsupported topics.
- An API session token protects subsequent steps but is not a full enterprise identity provider or a substitute for production authentication/authorization governance.
- Rate limiting and locks are demo-level; multi-worker race conditions, shared inventory atomicity and cross-replica session consistency need additional work.
- Voice requests add external processing; avoid sending real passenger information and keep keys server-side.
- Visual similarity and public Emirates asset use do not imply endorsement or affiliation.
- No pen test, production load test, external provider E2E validation or airline legal/security approval is asserted.

2.1.3. Standards

- Layered API architecture and REST-style endpoint contracts.
- Strict request validation through Pydantic; extra fields are rejected on strict models.
- Allowlisted action dispatch; deterministic hard gate before completion.
- Least privilege for optional storage, model and provider credentials; server-side secrets only.
- HTTPS for deployed user-facing endpoints and allowlisted external RAG source fetches.
- Data minimization, audit redaction, session TTL and conservative failure behaviour.
- Source provenance and authority/freshness checks for high-risk informational responses.
- Testable adapter contracts and mock-based unit/integration tests before any external integration.
- Dependency/secret scanning and reproducible frontend/backend CI where enabled.
- Emirates public check-in/booking references cited in Appendix A are subject to route, carrier, market and policy nuances; they do not turn local mock thresholds into airline policy.

2.2. ALIGNMENT WITH TARGET ARCHITECTURE

Component	Implementation in this repository	Architecture alignment / boundary
React + Vite	Booking portal, check-in workspace, floating chat, shared form state	Passenger experience; synthetic UI flow
FastAPI	API routes, Pydantic request/response models, session-aware endpoints	Application/API gateway
LangGraph orchestrator	Short allowlisted graph invocation; specialists behind handler dispatch	Bounded multi-agent workflow; no uncontrolled execution
Specialist services	Booking, eligibility, documents, seats, ancillaries, check-in, FAQ, conversation intent	Separation of concerns; deterministic decisions for transaction gates
Agentic RAG	LangGraph retrieval/grade/rewrite/authority/generate/evaluate/correct/abstain graph	Evidence-grounded passenger help, distinct from authorization
Mock Navitaire adapter	Seeded/demo booking and seat/ancillary operations; optional separate read-only diagnostics	Provider seam only; does not assert real Emirates system
Mock Timatic adapter	Synthetic document rules and result contract	Reference-style interface only; no official Timatic decision
SQLite / optional Chroma	Versioned FAQ source set with lexical/vector retrieval paths	Retrieval corpus and local semantic index
Optional search endpoint	SearXNG-compatible endpoint, trusted host allowlist, HTTPS fetch guard	Live public information only when explicitly configured
Redis / MongoDB	Optional session/checkpoint/audit storage with expiry	Persistence options; not proven multi-replica production semantics
Optional Neo4j	Rule/provenance graph	Explanatory relationships; not authoritative PASS source
Sarvam API	Server-side STT/TTS proxy	Optional voice I/O; keep secret on backend
GitHub Actions	Repository CI tests/build/scans	Change validation, not production certification

3. Current Architecture design

The following diagram is recreated for this project in the same high-level, multi-lane architecture-infographic format used in the supplied reference document. It shows the current POC components and flow boundaries; optional services are explicitly marked as optional, and the synthetic provider systems are not represented as real Emirates backend integrations.

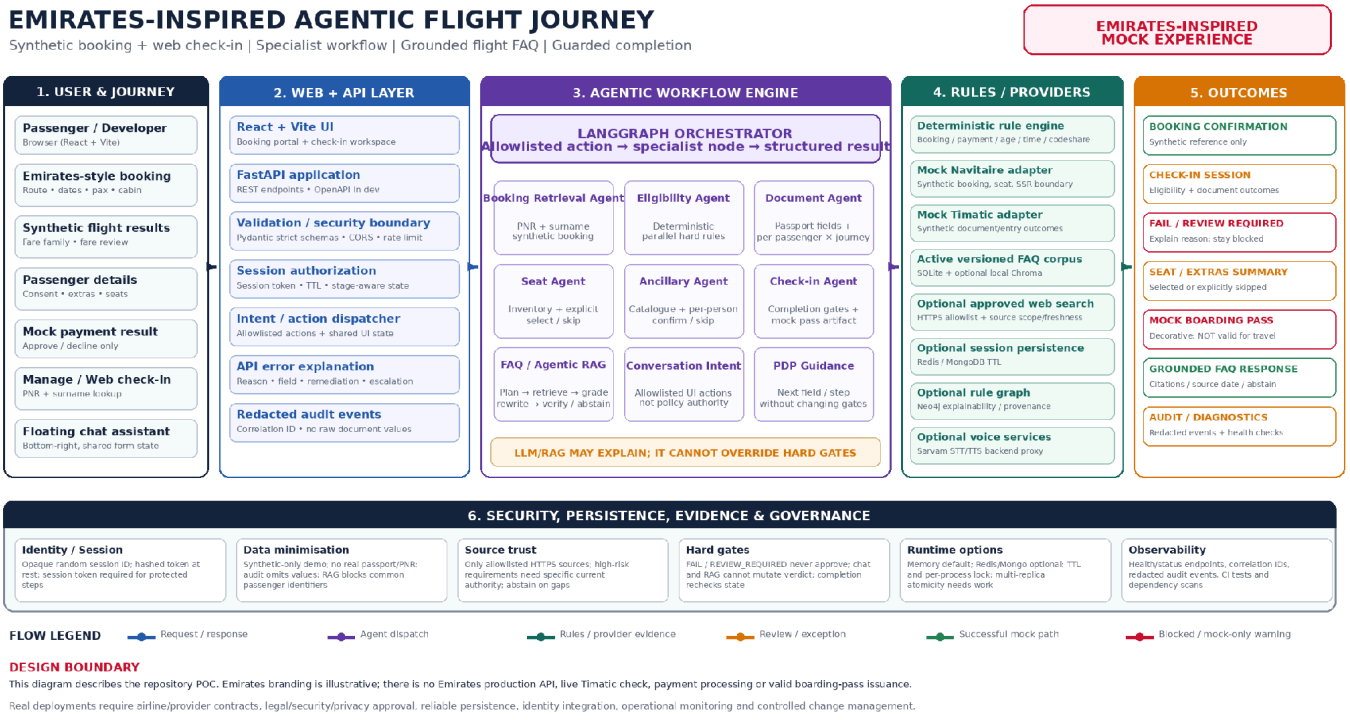


Figure 1. Emirates-inspired Agentic Mock Flight Website — logical solution architecture (repository-grounded).

Architecture components:

The architecture separates the passenger experience, validation/API boundary, specialist workflow, rules/provider abstractions, and outcomes. The workflow is stateful from the user perspective, but each request/action is dispatched as a bounded operation. The diagrams and component descriptions reflect the repository design and stated limitations rather than a proposed production integration to Emirates systems.

1. Passenger / Browser

Purpose:
Provide a single entry point for flight search, booking, Manage Booking, web check-in, FAQs and error assistance.
Role:
The browser renders React/Vite components. The in-page form and sidebar chatbot share UI state. The floating circular assistant launcher is positioned at the bottom-right and minimized when closed. All flight/seat/fare/payment choices are synthetic.

2. Booking Portal and Check-in Workspace

Purpose:
Model the traveller-facing airline workflow.
Role:
Booking proceeds through route/date/passenger search, synthetic flights and fare families, passenger/contact details and consent, per-passenger extras/seats, fare review, simulated payment and mock booking confirmation. Check-in retrieves a mock booking, runs gate checks, captures synthetic document data, completes optional choices and displays decorative mock pass artifacts.

\3. FastAPI Application Gateway

Purpose:
Centralize HTTP request handling and response contracts.
Role:
Routes validate request models, enforce expected session identifiers/tokens for protected passenger workflow endpoints, expose health/knowledge diagnostics, call commerce/workflow services, and return structured validation and business errors. OpenAPI is available in development. Production use requires deployment-specific identity, logging, TLS and policy controls.

4. Input Validation and Session Authorization

Purpose:
Reject malformed or unauthorized actions before they reach specialists.
Role:
Pydantic schemas constrain field lengths, patterns, enums, country codes, dates, passenger/journey keys and seat codes. The booking lookup normalizes the reference and surname, creates a random session token, stores only its hash with session state and returns the token to the browser. Session expiry and token validation protect later session actions; this is demo session authorization, not enterprise SSO.

5. LangGraph Orchestrator

Purpose:
Route one accepted action to the appropriate specialist operation.
Role:
The dispatcher maintains an allowlist of actions such as lookup, eligibility, document save/validation, seats, ancillaries, check-in completion and FAQ. Graph state is designed to contain an action, session/request identifiers and opaque result references; raw payload/results remain in one-call process memory. A deterministic fallback is supported in dependency-light execution.

6. Booking Retrieval Agent

Purpose:
Retrieve the passenger's synthetic booking and initialize the workflow session.
Role:
Validates reference + surname, retrieves the matching mock record, creates a random session ID/token and initializes stage, document decisions, seat assignments, extras, events and empty completion state. It returns a sanitized public session summary. In production this role would use an approved airline reservation contract and authenticated authorization model.

7. Eligibility Agent and Deterministic Rules

Purpose:
Evaluate the preconditions for continuing to check-in.
Role:
Pure rule tasks can check journey presence, booking status, payment status/balance, passenger name structure, age/accompaniment and other configured conditions concurrently. Results are structured with decision, rule ID, explanatory message and source type. Reference-derived rules remain flagged as mock policy until approved for a target carrier. LLM/RAG output does not determine the gate.

8. Passenger / Document Validation Agent

Purpose:
Validate user-entered synthetic travel-document fields and invoke the configured mock document contract.
Role:
Strict passport input validation covers number format, ISO alpha-2/alpha-3 nationality and issuing country, dates, purpose of stay and duration. DOB is required for designated passenger categories. The agent evaluates each passenger for each journey, calls MockTimaticAdapter, stores decision code and reasons, and aggregates conservatively. No real document authenticity, immigration or visa query is made by the end-user workflow.

9. Mock Timatic Adapter

Purpose:
Isolate a Timatic-style request/response contract from the agent logic.

Role:

Builds synthetic document and itinerary structures with a mock processing marker and returns PASS/FAIL/REVIEW_REQUIRED-like outcomes. This provides testability for mapping and user messaging. It must never be described as a live Timatic decision or a guarantee that the passenger may board or enter a country.

10. Seat Agent

Purpose:
Offer deterministic seat-map selection per passenger and itinerary leg.
Role:
The mock adapter returns seat codes, categories and illustrative prices. The agent checks passenger and journey keys, seat-code pattern, mock inventory availability and duplicate ownership. Explicit skip remains a separate state rather than silently random-assigning a seat. Cross-replica atomicity is not guaranteed by local process locks.

11. Ancillary Agent

Purpose:
Handle optional extras without confusing selection with confirmation.
Role:
The agent provides a synthetic extras catalogue, saves selected codes per passenger and requires a final confirmation or explicit no-extras/skip action as defined by the workflow. Real ancillary availability, prices, payment and fulfilment do not occur.

12. Check-in Agent and Completion Gate

Purpose:
Make the final mock completion decision after all mandatory checks.
Role:
Before returning pass artifacts, completion rechecks stage, eligibility, document results, seat/ancillary workflow state and applicable trip/person conditions. FAIL or REVIEW_REQUIRED blocks output. Any generated pass clearly states it is mock and not valid for travel. The agent cannot be told by natural language to bypass a server-side gate.

13. FAQ / Agentic RAG Agent

Purpose:
Answer general flight and travel questions from retrieved evidence and explain page errors.
Role:
The LangGraph RAG graph classifies the query, plans/decomposes searches, retrieves from the active versioned FAQ corpus and optional configured retrievers, grades relevance, rewrites weak queries within a bound, checks source trust/freshness for high-risk topics, generates a checked answer, evaluates support, optionally corrects it and abstains when evidence is insufficient. Citations/source metadata should be shown where available. It is not a travel authorization engine.

14. Knowledge Corpus and Source Catalog

Purpose:
Supply reviewed, versioned information for FAQ grounding.
Role:
Knowledge assets include the active SQLite FAQ corpus, `knowledge.json`, `official_source_catalog.json`, rule catalog and related ingestion scripts. Optional Chroma provides vector retrieval; lexical retrieval is the fallback. Sources cover general topics such as baggage, pets, assistance, disruptions, refunds and other FAQs but are not exhaustive for every route or jurisdiction. Policy and source dates require ongoing curation.

15. Optional Approved Live Search

Purpose:
Expand information retrieval beyond the local corpus where explicitly configured.
Role:
A SearXNG-compatible endpoint may return public results; source URLs must match an HTTPS allowlist before fetching. The service should not receive real passenger identifiers, PNRs or passport data. If this endpoint is configured and unavailable, the retrieval layer fails closed instead of silently switching corpora. A local `127.0.0.1` example is not reachable from a remote hosting platform.

16. Optional Azure OpenAI Model Provider

Purpose:
Improve query planning and evidence-grounded synthesis.
Role:
The backend can use an Azure OpenAI-compatible endpoint with configured endpoint, deployment and API version. Credentials stay in backend environment configuration. If model credentials are absent, a deterministic planning/source-extractive path may still answer from local evidence. A model response is checked and may be corrected or refused; it never overrides a workflow gate.

17. Session Store and LangGraph Checkpointer

Purpose:
Preserve short-lived journey state and graph continuity.
Role:
Memory is the local default. Optional Redis supports session/checkpoint storage; MongoDB can support session and/or checkpoint storage where configured. TTL is configurable. Session state includes synthetic booking and decisions needed for continuity, so it must be access-controlled and expired. The implementation notes call out that per-process locking is not a full multi-replica consistency solution.

18. Redacted Audit Sink

Purpose:
Create traceable operational events without duplicating passenger secrets.
Role:
Events include timestamp, component, action, outcome, correlation ID and hashed session fingerprint. Sensitive fields such as passport number, PNR/reference, names, DOB, password/token and authorization are redacted or excluded. MongoDB TTL collection is optional; local memory is the fallback. Audit messages must remain concise and must never include credentials or raw provider response payloads.

19. Optional Neo4j Rule Graph

Purpose:
Represent rule relationships/provenance for explainability.
Role:
The graph can describe rule catalogue relationships, dependencies and explanatory provenance. It is an optional support layer, not the source of truth for an eligibility verdict. Deterministic service output remains authoritative.

20. Voice Gateway (Optional)

Purpose:
Allow speech input and read-aloud responses.
Role:
Sarvam STT/TTS is proxied by backend endpoints and uses the same action dispatch as typed chat. API keys must remain server-side, mic use requires user permission and secure context, and audio latency depends on the synchronous REST path. Do not send actual travel-document values to speech/chat services.

21. Provider Diagnostic Workflow

Purpose:
Test an authorized reference-style read sequence without mutating bookings.
Role:
Opt-in protected diagnostics can probe supported staging reads and document-check requests where access is authorized. The reference workflow describes booking retrieval, rule prechecks, Timatic checks per passenger/journey and then a Navitaire passenger-document check only after explicit PASS. The diagnostic path must never update documents, seats or extras, finish check-in or issue a pass.

22. Error Explanation and Recovery

Purpose:
Make blocked workflows understandable and actionable.
Role:
The UI and assistant share page-error context. Error guidance maps codes such as expired passport, invalid country code, invalid dates, missing DOB, missing purpose/duration and unverified entry rules to reasons and safe next steps. Where current official requirements or authenticity checks are needed, the assistant must say so and maintain the blocked status.

23. Monitoring, CI and Release Evidence

Purpose:
Provide observability and repeatable validation.
Role:
The repository has backend test modules and GitHub Actions workflows for CI/build/secret-scan and documented dependency audit commands. Health endpoints expose relevant status. Tests with fake transport/mock providers establish contract behaviour only; they do not prove live service integration, production availability or policy compliance.

24. Output and Trust Boundary

Purpose:
Keep outputs distinguishable by trust level.
Role:
Outputs include synthetic booking confirmation, structured eligibility/document results, selected/skip seat and ancillary summary, FAQ text with evidence metadata, redacted audit event, and a mock boarding-pass card. Every user-facing flow must disclose the simulation boundary; public airline guidance may be summarized only with correct carrier/route scope and dated evidence.

4. Non-Functional Requirements

4.1. Archiving

The POC should retain only what is necessary to resume a synthetic session or troubleshoot a run. Session and audit TTL settings are configurable; MongoDB audit documents create an expiry index. Local memory is ephemeral. A production solution would require approved retention schedules, data residency, deletion, backup/restore, incident hold and privacy review.

Data item	Current / intended retention	Notes
Synthetic booking/session state	Default configured session TTL (repository default: 7 days)	May contain synthetic names, mock booking and input fields; protect with session authorization and expiry.
LangGraph checkpoint	Memory default; optional Redis/MongoDB with configured TTL	Avoid raw passenger request payloads in graph state.
Audit events	Configurable TTL (repository default: 7 days when DB enabled)	Redacted component/action/outcome; raw passenger values should not be recorded.
FAQ corpus / source catalog	Versioned assets until replaced by approved revision	Track source URL, reviewed version and content timestamp; refresh after policy changes.
Chroma index	Local generated cache/index	Can be regenerated from active approved corpus; do not treat as authoritative original data.
CI logs / test artifacts	GitHub repository/workflow retention settings	Do not log credentials, real travel documents or full request bodies.

4.2. Logging

Logging should support triage without reproducing passenger data. Include workflow name/action, timestamp, safe error category, duration, correlation ID, agent/component, external service outcome category, retry count and redacted session fingerprint.

- Never print API keys, access tokens, authorization headers, passport number, PNR, full passenger name, DOB or raw document/request body.
- Log the distinction between MOCK_ONLY, local corpus, live public search and provider diagnostics.
- Record retrieval corpus version/source identifiers to reproduce FAQ answers.
- Log when high-risk RAG abstains due to evidence gaps rather than emitting generic “success”.
- Mask provider error bodies before surfacing them to the browser.

4.3. Auditing

The following events should be auditable: booking lookup success/failure, session creation/expiry, eligibility evaluation and failed rule IDs, document validation per passenger/journey (without raw values), seat selection/skip, ancillary selection/confirmation/skip, completion gate result, RAG retrieval/version and abstention, provider diagnostic call, configuration changes, and administrative access to optional stores.

Audit records should use a random/correlated session fingerprint rather than PNR as the lookup key. Source-corpus changes should record the version label, change note, source identifiers and activation time.

4.4. Scalability

For the current POC, scaling is constrained by the chosen process model, synthetic inventory, configured model/search quotas and optional external services. Horizontal web scaling cannot be assumed safe until session persistence and seat ownership are shared with atomic transactional semantics and concurrent updates are tested.

- Make API processes stateless apart from shared session/inventory persistence before multi-replica deployment.
- Use idempotency keys for booking confirmation and action retries before real transactions are introduced.
- Bound RAG query fan-out, result count, source-page fetches, model tokens and correction attempts.
- Use backpressure and concurrency limits for optional model, web-search and provider diagnostics.
- Keep real provider transactions behind separate feature flags and explicit credentials; mock mode remains the default.

4.5. Rate Limit

The API has demo-level rate limiting. Deployed services should independently limit by session, IP/identity and route; protect expensive RAG and voice endpoints; set provider timeouts; and use bounded retry/backoff only for transient faults.

Surface	Required controls
FastAPI public routes	Rate limits; request-size caps; strict schemas; CORS allowlist; HTTPS; suspicious pattern blocking.
LLM / RAG	Max retries; query/result/token bounds; timeouts; concurrency limits; high-risk abstention.

Surface	Required controls
Live search / source fetch	HTTPS host allowlist; redirect validation; page count/size limits; timeout; no PII in queries.
Voice STT/TTS	Audio duration/size caps; per-session quotas; server-side API key; no silent repeated billing loops.
Provider diagnostics	Disabled by default; secret/header auth; allowlisted endpoints/carrier; read-only operations only.
Storage	TTL, connection pool bounds, timeouts and fail-fast when explicitly configured but unavailable.

4.6. Performance

No production performance SLA has been tested. The app includes asynchronous API operations and can use a local corpus without a remote model, but latency varies by corpus retrieval, optional semantic store, web fetch, model, voice service and database availability. Targets below are engineering objectives for load-test planning, not measured service levels.

Metric	Proposed objective for POC validation	Measurement notes
Booking form/local validation	p95 < 300 ms, excluding search/render network	Measure on hosted test backend with synthetic data.
Synthetic lookup and eligibility	p95 < 1 s for local/mock mode	Include session-store round trip and deterministic checks.
Document checks	p95 < 2 s for mock passenger × journey fan-out	Report per-case and aggregate latency; cap fan-out.
Local FAQ retrieval only	p95 < 2 s at normal demo corpus size	Separate retrieval from LLM generation.
RAG with model/live search	No fixed target until provider quotas are known	Track each stage and show a safe timeout response.
Voice STT/TTS	No SLA; display pending state and allow text fallback	Current REST implementation is non-streaming.
Complete booking/check-in demo flow	Define journey success and total duration in test plan	Do not claim airline transaction latency.
Hosted stability	Documented test concurrency and error budget before launch	Must include multi-worker and race tests.

4.7. System Management and Monitoring

- API health, readiness and startup-config validation; distinguish optional integration unconfigured from configured-but-unavailable.
- Request rate, error rate, p50/p95 latency, 4xx/5xx counts and rejected payload classes.
- Session creation/expiry, unauthorized access attempts, expired sessions and persistence failures.
- Agent completion, failed gate counts, result-validation failures and graph fallback mode.
- RAG retrieval mode, active corpus version, source count/freshness, search failures, rewrite count, evidence grade, correction count and abstention reason.
- Model calls, latency, token usage, quota failures and configuration status without logging prompt PII.
- Provider diagnostic invocations and safe status; no live connectivity claim unless a real authorized test was performed.
- Storage health for Redis/MongoDB/Neo4j/Chroma and TTL/index diagnostics.
- CI test/build/secret-scan/dependency-scan result plus exact revision and runtime used.
- Alert on sustained auth failures, elevated review-required output, provider/search failure, RAG citation loss or unusually high retry/token use.

4.8. User Role and Access Management

Role	Responsibility / access principle
Passenger / demo user	Use synthetic bookings; access only own demo session using protected session token.
Developer	Maintain code and tests; no production secrets in source or frontend environment.
Platform / DevOps maintainer	Manage hosting, environment variables, TLS, logs, secret rotation, deploy and rollback.
Knowledge curator	Review official source URLs, carrier scope, last-checked date, rule summaries and corpus activation.
Security / privacy reviewer	Review PII handling, token lifetimes, data retention, external processors, threat model and access logs.
Provider integration owner	Hold explicitly approved Navitaire/Timatic-like provider access; maintain read-only diagnostics and contract tests.
Airline policy / legal stakeholder	Decide which official rules and workflows may be adopted; approve any production claims or route-specific treatment.

Controls: least privilege; server-side secrets; no credentials in `VITE_*`; HTTPS; short-lived sessions; token hashing at rest; audit redaction; dependency/secret scan; restricted diagnostic routes; explicit consent for any real-data test; separate dev/test/prod configuration; no public access to storage; and documented retention. The POC does not itself implement full SSO, organizational RBAC or production incident response.

5. Risk and Mitigation

Risk	Impact	Mitigation / acceptance condition
POC mistaken for actual Emirates service	User trusts mock booking or boarding pass	Place mock-only disclaimer at entry, confirmation, provider checks and pass output; keep no real payment or airline mutation.

Risk	Impact	Mitigation / acceptance condition
Local demo rules mistaken for current entry rules	Incorrect travel advice or boarding/entry outcome	Use official dated route-specific sources; require authorized Timatic/immigration decision for production; RAG abstains when evidence is weak.
Model hallucination / prompt injection	Unsafe or misleading response	Separate LLM from decision gates; action allowlist; source grading; authority checks; output validation; fail closed on high risk.
PII in logs/checkpoints/search	Privacy breach	Synthetic data only, payload minimization, redacted audit, opaque graph references, session TTL, search input blocking.
Session token theft or cross-session access	Unauthorized use of passenger flow	HTTPS, secure browser storage strategy, token rotation/expiry, no token in logs/URLs, test cross-session isolation.
Single-process persistence and race conditions	Lost state or conflicting seat assignments	Shared transactional store, optimistic locking/idempotency, multi-worker/load/race tests before horizontal scale.
Optional database configured but unreachable	Startup/runtime failures or lost state	Fail-fast on explicitly configured integration; readiness checks; alert and document local fallback limits.
Search endpoint unavailable or untrusted	Incomplete or manipulated facts	HTTPS allowlists, redirects guarded, retrieval mode shown, timeouts, do not silently switch backend; abstain.
Stale airline public guidance	Incorrect date/time policy	Record checked date/source URLs and refresh schedule; distinguish carrier policy from jurisdiction rules.
Mock provider contract differs from actual provider	Integration rejection or wrong mapping	Use approved current provider specs, contract tests and a sandbox validation plan; never infer Emirates provider choice.
Unbounded costs/latency from RAG/voice	Quota exhaustion and slow UX	Bound query fan-out/retries, provider timeouts, quotas, rate limits and text fallback.
Public demo exposure	Abuse or scraping	Rate limiting, CORS/HTTPS, abuse monitoring, synthetic-only fixtures, diagnostic token controls, security review.
Visual branding implies affiliation	Trademark/endorsement confusion	Label “Emirates-inspired mock”; follow asset usage/brand guidelines and obtain permission if necessary.
Inconsistent docs/CI evidence	Overstated readiness	Pin a commit SHA in release documentation; rerun current test/build/audit and state exact results.
Provider diagnostics change data by mistake	Booking/document/seat mutations	Keep diagnostic routes protected, opt-in and read-only; integration tests assert no mutation/pass generation.

6. Appendices

Appendix A – REFERENCES

Repository documentation was inspected for this design, and public airline rules below were cross-checked on 12 October 2026. Links should be revalidated before production use because route, market and carrier rules can change.

1. **1. Project repository —** https://github.com/AIstar007/Agentic-Mock-Flight-Website
2. **2. Architecture and decisions —** https://github.com/AIstar007/Agentic-Mock-Flight-Website/blob/main/docs/ARCHITECTURE.md
3. **3. Video workflow implementation notes —** https://github.com/AIstar007/Agentic-Mock-Flight-Website/blob/main/docs/VIDEO_WORKFLOW_IMPLEMENTATION.md
4. **4. Validation results and limits —** https://github.com/AIstar007/Agentic-Mock-Flight-Website/blob/main/docs/VALIDATION_RESULTS.md
5. **5. Provider diagnostic boundaries —** https://github.com/AIstar007/Agentic-Mock-Flight-Website/blob/main/docs/PROVIDER_DIAGNOSTICS.md
6. **6. Adapter contracts —** https://github.com/AIstar007/Agentic-Mock-Flight-Website/blob/main/docs/ADAPTER_CONTRACTS.md
7. **7. Official Emirates homepage / service navigation —** https://www.emirates.com/in/english/
8. **8. Emirates Manage Booking: reference + last name, itinerary, extras and seats —** https://www.emirates.com/english/manage-booking/
9. **9. Emirates online check-in timing and exceptions —** https://www.emirates.com/uk/english/help/faq-topics/preparing-to-travel/
10. **10. Emirates online check-in entry —** https://www.emirates.com/in/english/manage-booking/online-check-in/
11. **11. Emirates seat selection —** https://www.emirates.com/english/book/about-booking-online/emirates-seat-selection/
12. **12. Emirates dangerous goods policy —** https://www.emirates.com/english/before-you-fly/travel/dangerous-goods-policy/
13. **13. Emirates online booking basics and advance booking window —** https://www.emirates.com/english/help/faq-topics/booking-with-emirates/
14. **14. SearXNG official setup documentation —** https://docs.searxng.org/admin/installation-docker.html
15. **15. Microsoft Azure OpenAI / Foundry data privacy —** https://learn.microsoft.com/en-us/azure/ai-foundry/responsible-ai/openai/data-privacy
16. **16. Sarvam STT API reference (repository README reference) —** https://docs.sarvam.ai/api-reference/speech-to-text/transcribe
17. **17. Sarvam TTS API reference (repository README reference) —** https://docs.sarvam.ai/api-reference/text-to-speech/convert

Verified Emirates reference facts
The official preparing-to-travel FAQ states online check-in is available from 48 hours until 90 minutes before scheduled departure for eligible e-ticket passengers, and online check-in may be unavailable if the first itinerary flight is not operated by Emirates. The public Manage Booking page asks for last name and booking reference. These are external reference facts, not implemented live-rule guarantees.

Appendix B – GENERATED FILES / REPOSITORY ASSETS

Asset / path	Purpose
README.md	Project scope, setup, limitations, endpoints, RAG configuration and deployment notes.
docs/ARCHITECTURE.md	Logical topology, separation of agents/rules/providers and hard-gate safety contract.
docs/VIDEO_WORKFLOW_IMPLEMENTATION.md	Booking/check-in user journey, UI state, chatbot behaviour and workflow-validation notes.
docs/VALIDATION_RESULTS.md	Tests believed completed at documented revision and explicit non-verified items.
docs/ADAPTER_CONTRACTS.md	Navitaire/Timatic adapter boundary documentation.
docs/PROVIDER_DIAGNOSTICS.md	Protected read-only provider diagnostic configuration and caveats.
docs/REFERENCE_WORKFLOW_MIGRATION.md	Reference-style read-only validation ordering and migration notes.
docs/EVALUATION_PLAN.md	Adversarial and quality-gate test proposals.
backend/app/main.py	FastAPI routes, dependency setup and API workflow functions.
backend/app/agents.py	Specialist deterministic workflow services and public session projection.
backend/app/orchestrator.py	LangGraph dispatcher, action allowlist and graph persistence setup.
backend/app/agentic_rag.py	Self-corrective RAG planner/retriever/authority/evaluation/correction flow.
backend/app/rag_agentic.py	RAG retrieval/synthesis support layer.
backend/app/commerce.py	Synthetic flight search, passenger/booking validation and booking confirmation.
backend/app/rules.py	Deterministic eligibility rules and parallel rule evaluation.
backend/app/adapters/navitaire.py	Mock booking / seat / ancillary provider boundary.
backend/app/adapters/timatic.py	Mock travel-document requirement adapter.
backend/app/state_store.py	Memory / Redis / Mongo session persistence options and TTL.
backend/app/audit.py	Redacted audit event sink and optional MongoDB TTL storage.
backend/data/knowledge.json	Reviewed/source-oriented knowledge records for retrieval/ingestion.
backend/data/official_source_catalog.json	Source catalog and source provenance metadata.
backend/data/rules_catalog.json	Rule catalog for deterministic eligibility/reference rules.
backend/data/faq_corpus.sqlite3	Activated local versioned FAQ corpus (not exhaustive legal database).
backend/tests/*.py	Automated tests for API, commerce, FAQ storage, provider diagnostics and reference workflow.
frontend/package.json	React/Vite dependencies and scripts.
.github/workflows/ci.yml	CI workflow (backend tests/frontend build, according to current file configuration).
.github/workflows/secret-scan.yml	Secret scanning workflow configuration.

Appendix C – API surface (high-level; confirm exact schema in OpenAPI)

Endpoint family	Intent / note
POST /api/v1/bookings/search	Synthetic route/date/passenger/cabin search.
POST /api/v1/bookings/confirm	Create a synthetic booking after simulated payment result.
GET /api/v1/flight-status	Deterministic illustrative flight status, not live operations data.
POST /api/v1/pdp/context	Optional passenger-form next-field guidance.
POST /api/v1/site-assistant/chat	Flight-scoped FAQ and allowlisted UI actions.
GET /api/v1/rules	Rule catalog / optional Neo4j read.
POST /api/v1/bookings/lookup	PNR + surname lookup; creates session token.
GET /api/v1/sessions/{sid}	Authorized public session summary.
POST /api/v1/sessions/{sid}/eligibility	Deterministic booking rule checks.
POST /api/v1/sessions/{sid}/documents	Save synthetic passport fields.
POST /api/v1/sessions/{sid}/documents/validate	Mock per passenger/per journey document checks.
GET/POST /api/v1/sessions/{sid}/seats	Mock seat map / mock assignment or skip.
GET/POST /api/v1/sessions/{sid}/ancillaries	Catalogue / passenger selection.
POST /api/v1/sessions/{sid}/ancillaries/confirm	Explicitly confirm extras or no-extras.
POST /api/v1/sessions/{sid}/check-in	Generate mock pass only if every hard gate passes.
GET /api/v1/sessions/{sid}/events	Redacted events.
POST /api/v1/chat	FAQ and allowlisted chat actions.
POST /api/v1/provider-tests/*	Optional protected read-only provider diagnostics; disabled unless configured.

Appendix D – Key configuration groups (names only; no secrets)

Group	Representative environment configuration	Security / operational note
Runtime	APP_ENV, MOCK_MODE, FRONTEND_ORIGIN, REQUIRE_PERSISTENT_STORAGE	Keep MOCK_MODE=true for the public demo; frontend origin must be exact.
Session / audit TTL	SESSION_TTL_SECONDS, AUDIT_TTL_SECONDS	Tune per environment and retention policy.

Group	Representative environment configuration	Security / operational note
Persistence	REDIS_URL, LANGGRAPH_REDIS_URL, MONGODB_URI, MONGODB_DATABASE	Secrets only on backend; test failure mode and TTL indexes.
Rule graph	NEO4J_URI, NEO4J_USERNAME, NEO4J_PASSWORD, NEO4J_REQUIRED	Optional explanatory graph, not verdict authority.
RAG search	RAG_WEB_SEARCH_URL, RAG_WEB_ALLOWED_HOSTS, RAG_WEB_FETCH_PAGES, RAG_MAX_RETRIES, RAG_TOP_K	Use an approved, reachable endpoint; never include passenger data.
Azure OpenAI	AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT, AZURE_OPENAI_API_VERSION, AZURE_OPENAI_API_KEY	Server-side only; verify model provider/region/processing/privacy requirements.
Voice	SARVAM_API_KEY, SARVAM_STT_MODEL, SARVAM_TTS_MODEL, SARVAM_TIMEOUT_SECONDS	Server-side only; disclose third-party processing.
Provider diagnostics	ENABLE_PROVIDER_DIAGNOSTICS, PROVIDER_DIAGNOSTICS_TOKEN, NAVITAIRE_*, TIMATIC_*	Off by default; actual access contracts required; diagnostic route remains read-only.

Appendix E – Verification matrix and open decisions

Area	Reported status in repository documents	Before any production claim
Backend logic	Docs report multiple passing pytest runs (70/84/88/92 in different dated notes); latest note reports 92 backend tests passed on 11 Oct 2026. These are not independently rerun for this document.	Run current commit test suite and archive revision, command and complete output.
Frontend	README/video notes report Vite production build success on Node 22.x; package requires >=22.12.0.	Run npm ci + npm run build against the current lockfile and deployment target.
Live Navitaire / Timatic	Not verified live; test suites use mocks/fake HTTP transport. End-user workflow uses mock adapters.	Get written provider access, confirm contract/schema and execute approved sandbox tests.
Emirates backend system choice	Not established by source; Navitaire is only an adapter boundary.	Do not infer actual reservation/DCS provider; architecture must be revalidated with airline owner.
RAG web search	Optional and must be configured; local corpus is the offline/default knowledge boundary.	Test allowlist, redirects, page freshness and hosted network reachability.
Chroma / Redis / MongoDB / Neo4j	Optional; external integration is not comprehensively verified in all notes.	Run environment-specific integration, TTL, outage and race tests.
Security / dependencies	Repo notes say pip/npm audit, SBOM, image scan and penetration test were not completed at the documented point.	Run current dependency audit, SBOM, secret scan, threat modelling and independent penetration test.
Compliance / legal	No certification/airline approval claimed.	Obtain privacy/legal, aviation security, airline operations, accessibility and provider agreements before production.
Operational scale	No formal load, browser matrix or multi-worker race test claimed.	Establish concurrency targets, SLOs, alerting, incident/rollback, disaster recovery and support ownership.
Rules corpus	Curated and versioned, not exhaustive across country rules.	Assign content owner, review cadence, authority hierarchy, expiry notifications and rule-change release process.

Release decision
Suitable as a synthetic design/engineering POC when all mock-only boundaries remain visible. Not ready to claim production Emirates booking/check-in capability until the listed integration, rule-governance, security, privacy, resilience and approval work is completed.